

Configuración y Despliegue del Servicio

Sistema de Registros EAFIT - Práctica Final

Autores: Cristian Cabarcas, David Ruiz , David Quintero

Descripción General

Sistema web completo para registrar estudiantes interesados en programas académicos de la Universidad EAFIT. La aplicación incluye:

- **Aplicación web bilingüe** (Español e Inglés) con formularios de registro
- **Balanceador de carga** con NGINX (Round Robin)
- **Base de datos MySQL 8.0** con soporte completo UTF-8
- **Dashboard de estadísticas** con gráficas interactivas
- **Sistema de reportes** por email
- **SSL/TLS seguro** con certificados ACM
- **Orquestación con Docker** para desarrollo local
- **Despliegue en AWS** con ECS Fargate, RDS y ALB

Características Principales



Formulario de Registro

- Nombre completo (validado)
- Selección de comuna (10 opciones)
- Selección de carrera (Medicina, Ingeniería, Abogacía, Licenciatura)
- Soporte multiidioma (Español e Inglés)
- Mensajes de confirmación
- Validación cliente y servidor

Dashboard de Estadísticas

- Resumen general de registros
- Gráficas por comuna (barras)
- Distribución por carrera (pie charts)
- Análisis cruzado (comuna × carrera)
- Tablas detalladas con datos
- Envío de reportes por email
- Autenticación con contraseña

Sistema de Reportes

- Envío automático de reportes por email
- Contenido HTML formateado
- Gráficas incluidas en el correo
- Email destinatario configurable
- Integración SMTP Gmail

Diseño

- Tema moderno negro/morado
- Responsive (móvil, tablet, desktop)
- Gradientes y efectos visuales
- Animaciones fluidas
- Interfaz intuitiva

Requisitos del Sistema

Hardware

Desarrollo Local:

- Mínimo 2 vCPU
- 4GB RAM
- 20GB almacenamiento disponible

Producción (AWS):

- AWS EC2 t3.medium o superior
- ALB (Application Load Balancer)
- RDS db.t3.micro

Software

Componente	Versión	Propósito
-----	-----	-----
Docker	20.10+	Contenedores
Docker Compose	2.0+	Orquestación local
AWS CLI	2.0+	Gestión AWS
Python	3.11+	Runtime
MySQL	8.0+	Base de datos
Node.js	18+	Opcional (herramientas frontend)

Servicios Externos

1. Cuenta AWS (Amazon Educate recomendado)

- Acceso a ECR, ECS, RDS, ALB
- Crédito \$100 USD/mes para Educate

2. Dominio DuckDNS(GRATIS)

- <https://www.duckdns.org>

- Sin costo
- Soporte HTTPS

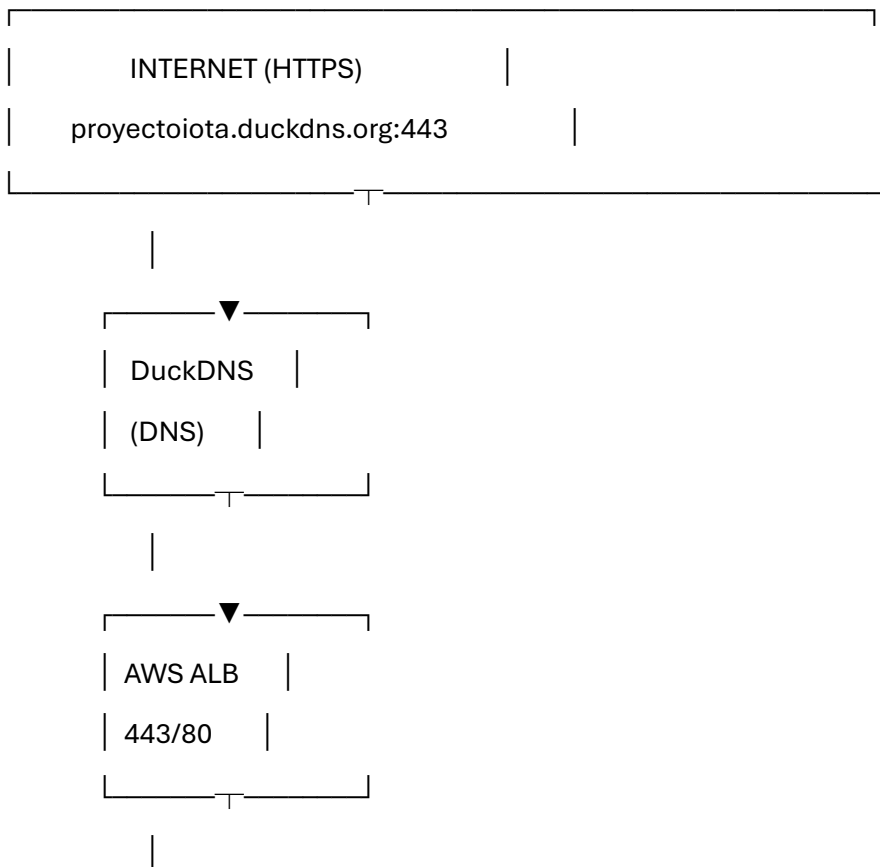
3. **Cuenta Gmail** con contraseña de aplicación

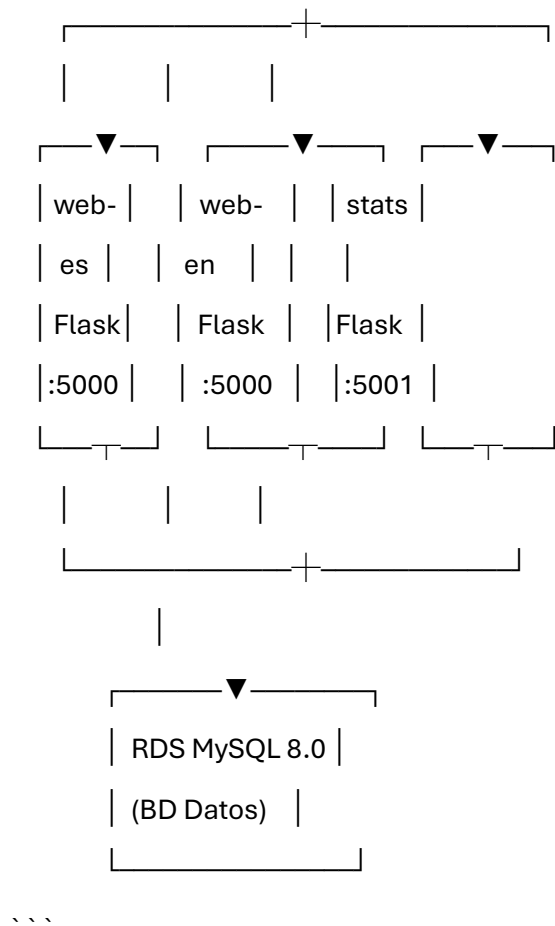
- Para SMTP (envío de reportes)
- Configuración fácil

Arquitectura del Servicio

Diagrama General

...





Componentes

Componente	Tecnología	Puerto	Función
Web-ES	Flask 3.0	5000	Formulario Español
Web-EN	Flask 3.0	5000	Formulario Inglés
Stats	Flask 3.0	5001	Estadísticas
NGINX	1.25-alpine	80/443	Balanceador
MySQL	8.0	3306	Base de datos
DuckDNS	DNS	DNS	Dominio
ALB	AWS	80/443	Load Balancer

Componentes Principales

1. Servidor Web de Registro (app-web)

Propósito: Formulario de registro de estudiantes

Especificaciones:

- **Framework:** Flask 3.0.0
- **Servidor:** Gunicorn 4 workers, 120s timeout
- **Puertos:** 5000 (HTTP interno)
- **Versiones:** web-es (Español), web-en (Inglés)
- **Base de datos:** PyMySQL 1.1.0 con charset UTF-8mb4

Funcionalidades:

- Renderizar formularios bilingües
- Validar datos de entrada
- Guardar registros en BD
- Retornar JSON con resultado
- Health checks

Dependencias:

```\n

Flask==3.0.0

PyMySQL==1.1.0

python-dotenv==1.0.0

cryptography==41.0.7

Werkzeug==3.0.0

Gunicorn==21.2.0

```\n

2. Servidor de Estadísticas (app-estadisticas)

Propósito: Dashboard y reportes

Especificaciones:

- **Framework:** Flask 3.0.0
- **Puerto:** 5001 (HTTP interno)
- **Servidor:** Gunicorn 4 workers
- **Visualización:** Matplotlib 3.8.2

Funcionalidades:

- Generar gráficos (barras, pie charts)
- Calcular estadísticas
- Mostrar dashboard HTML
- Enviar reportes por email
- API JSON de estadísticas
- Autenticación con contraseña

Dependencias:

```\n

Flask==3.0.0

Matplotlib==3.8.2

PyMySQL==1.1.0

python-dotenv==1.0.0

cryptography==41.0.7

Gunicorn==21.2.0

````

3. Proxy Inverso y Balanceador (NGINX)

Propósito: Balanceo de carga y proxy reverso

Especificaciones:

- **Versión:** 1.25-alpine
- **Puertos:** 80 (HTTP), 443 (HTTPS)
- **Balanceo:** Round Robin
- **Algoritmo:** Distribución equitativa

Configuración:

```
````nginx

upstream app_es {
 server web-es:5000;
}

upstream app_en {
 server web-en:5000;
}

upstream estadisticas {
 server stats:5001;
}

server {
 listen 80;
 server_name _;
```



```
location /es {
 proxy_pass http://app_es;
}

location /en {
 proxy_pass http://app_en;
}

location /stats {
 proxy_pass http://estadisticas;
}

location /api {
 proxy_pass http://estadisticas;
}
}
```

#### **Funcionalidades:**

- Balanceo automático
- Health checks
- Terminación SSL/TLS (en AWS)
- Headers de seguridad
- Compresión gzip

#### **4. Base de Datos (MySQL)**

**Propósito:** Almacenar datos de registros

### Especificaciones:

- **Versión:** 8.0
- **Charset:** utf8mb4 (soporte acentos)
- **Collation:** utf8mb4\_unicode\_ci
- **Almacenamiento:** Volumen persistente (db\_data)

### Tabla: registros

```sql

```
CREATE TABLE registros (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(255) NOT NULL,  
  comuna VARCHAR(100) NOT NULL,  
  carrera VARCHAR(100) NOT NULL,  
  fecha DATETIME NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP  
) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

```

### Datos de Ejemplo:

```

| id | nombre | comuna | carrera | fecha |
|----|-----------------|----------|------------|------------|
| 1 | María García | Comuna 1 | Ingeniería | 2026-05-24 |
| 2 | Andrés López | Comuna 2 | Abogacía | 2026-05-24 |
| 3 | Sofía Rodríguez | Comuna 3 | Medicina | 2026-05-24 |

```

---

## Configuración Local

### Paso 1: Clonar o Descargar Proyecto

```
```bash
cd "d:\Users\Cristian\Documents\Visual Projects\PracticaFinal"
```
```

### Paso 2: Configurar Variables de Entorno

Copiar archivo de ejemplo:

```
```bash
cp .env.example .env
```
```

Editar `.env` con tus valores:

```
```env
# Base de datos
DB_HOST=db
DB_USER=root
DB_PASSWORD=eafit_2025_secure
DB_NAME=usuarios
```

```
# Administración
```

```
ADMIN_PASSWORD=admin123
```

```
# Email (SMTP Gmail)
```

```
SMTP_SERVER=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=tu-email@gmail.com
SMTP_PASSWORD=tu-app-password-gmail
```

```
# Aplicación
FLASK_ENV=development
FLASK_DEBUG=1
` ``
```

Paso 3: Iniciar Servicios

Construir imágenes Docker

```
` `` bash
docker-compose build
` ``
```

Iniciar servicios en background

```
` `` bash
docker-compose up -d
` ``
```

Esperar inicialización (30-60 segundos)

```
` `` bash
# Verificar estado
docker-compose ps

# Ver logs en tiempo real
docker-compose logs -f
```

```

#### **Paso 4: Acceder a la Aplicación**

| Servicio      | URL                                      | Descripción        |
|---------------|------------------------------------------|--------------------|
| -----         | -----                                    | -----              |
| Registro (ES) | http://localhost/es                      | Formulario español |
| Registro (EN) | http://localhost/en                      | Formulario inglés  |
| Estadísticas  | http://localhost/stats?password=admin123 | Dashboard          |
| API Stats     | http://localhost/api/statistics          | JSON API           |
| Health Check  | http://localhost/health/es               | Verificación       |

#### **Paso 5: Verificar Funcionamiento**

##### **Probar registro**

```
```bash
curl -X POST http://localhost/register \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "nombre=Test User&comuna=Comuna 1&carrera=Ingeniería"
```
```

##### **Ver estadísticas**

```
```bash
curl http://localhost/stats?password=admin123
```
```

##### **Enviar email (si está configurado)**

```
```bash
```

```
curl -X POST "http://localhost/api/send-report?password=admin123&email=test@example.com"
```

```
```
```

## **Parar Servicios**

```
```bash
```

```
# Detener servicios
```

```
docker-compose down
```

```
# Detener y eliminar volúmenes (¡cuidado: borra datos!)
```

```
docker-compose down -v
```

```
```
```

```

```

## **Despliegue en AWS**

### **Opción 1: Despliegue Automatizado (Recomendado)**

#### **Requisitos Previos**

- AWS CLI 2.0+
- Credenciales AWS configuradas
- Acceso a AWS Educate (opcional pero recomendado)

#### **Ejecutar Script de Despliegue**

```
```powershell
```

```
# Cambiar a directorio del proyecto
```

```
cd "d:\Users\Cristian\Documents\Visual Projects\PracticaFinal"
```

```
# Permitir ejecución de scripts
```

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process
```

```
# Ejecutar despliegue completo
```

```
.\aws-deploy.ps1 -Action setup-all
```

```
# Verificar estado
```

```
.\aws-deploy.ps1 -Action status
```

```
` `` `
```

Opción 2: Despliegue Manual

Paso 1: Configurar AWS CLI

```
` `` `powershell
```

```
# Instalar AWS CLI
```

```
choco install awscli
```

```
# Configurar credenciales
```

```
aws configure
```

```
# Ingresar: Access Key, Secret Key, Region: us-east-1, Output: json
```

```
# Verificar configuración
```

```
aws sts get-caller-identity
```

```
` `` `
```

Paso 2: Crear Repositorios ECR

```
` `` powershell

# Crear repositorio para web-es
aws ecr create-repository `
  --repository-name practicafinal-web-es `
  --region us-east-1

# Crear repositorio para web-en
aws ecr create-repository `
  --repository-name practicafinal-web-en `
  --region us-east-1

# Crear repositorio para stats
aws ecr create-repository `
  --repository-name practicafinal-stats `
  --region us-east-1
` ``
```

Paso 3: Construir y Subir Imágenes

```
` `` powershell

# Obtener ID de cuenta
$ACCOUNT_ID = aws sts get-caller-identity --query 'Account' --output text
$ECR_REGISTRY = "$ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com"

# Iniciar sesión en ECR
aws ecr get-login-password --region us-east-1 | `
  docker login --username AWS --password-stdin $ECR_REGISTRY

# Construir imágenes
```


docker-compose build

Etiquetar imágenes

docker tag practicafinal-web-es:latest "\$ECR_REGISTRY/practicafinal-web-es:latest"

docker tag practicafinal-web-en:latest "\$ECR_REGISTRY/practicafinal-web-en:latest"

docker tag practicafinal-stats:latest "\$ECR_REGISTRY/practicafinal-stats:latest"

Subir a ECR

docker push "\$ECR_REGISTRY/practicafinal-web-es:latest"

docker push "\$ECR_REGISTRY/practicafinal-web-en:latest"

docker push "\$ECR_REGISTRY/practicafinal-stats:latest"

Verificar

aws ecr describe-repositories --region us-east-1

```

#### **Paso 4: Crear Base de Datos RDS**

```powershell

Definir contraseña

\$DB_PASSWORD = "EafitDB2025Secure!"

Crear instancia RDS

aws rds create-db-instance `

--db-instance-identifier eafit-mysql-db `

--db-instance-class db.t3.micro `

--engine mysql `

--engine-version 8.0.35 `

--master-username admin `

```
--master-user-password $DB_PASSWORD `
--allocated-storage 20 `
--publicly-accessible true `
--backup-retention-period 7 `
--region us-east-1
```

Esperar disponibilidad (5-10 minutos)

```
aws rds describe-db-instances `
--db-instance-identifier eafit-mysql-db `
--region us-east-1 `
--query 'DBInstances[0].[DBInstanceStatus,Endpoint.Address]'
` ` `
```

Paso 5: Crear ECS Cluster

```
` ` ` powershell
```

Crear cluster

```
aws ecs create-cluster `
--cluster-name eafit-cluster `
--region us-east-1
```

Registrar definición de tarea

```
aws ecs register-task-definition `
--cli-input-json file://ecs-task-definition.json `
--region us-east-1
` ` `
```

Paso 6: Crear Application Load Balancer

```
` `` powershell
```

```
# Obtener VPC ID
```

```
$VPC_ID = aws ec2 describe-vpcs `
  --filters "Name=isDefault,Values=true" `
  --query 'Vpcs[0].VpcId' `
  --output text
```

```
# Crear Security Group
```

```
$ALB_SG = aws ec2 create-security-group `
  --group-name eafit-alb-sg `
  --description "Security Group para ALB" `
  --vpc-id $VPC_ID `
  --query 'GroupId' `
  --output text
```

```
# Permitir HTTP
```

```
aws ec2 authorize-security-group-ingress `
  --group-id $ALB_SG `
  --protocol tcp `
  --port 80 `
  --cidr 0.0.0.0/0
```

```
# Permitir HTTPS
```

```
aws ec2 authorize-security-group-ingress `
  --group-id $ALB_SG `
  --protocol tcp `
  --port 443 `
  --cidr 0.0.0.0/0
```

```
# Obtener subnets
```

```
$SUBNETS = aws ec2 describe-subnets `
  --filters "Name=vpc-id,Values=$VPC_ID" `
  --query 'Subnets[0:2].SubnetId' `
  --output text
```

```
# Crear ALB
```

```
aws elbv2 create-load-balancer `
  --name eafit-alb `
  --subnets $SUBNETS.Split() `
  --security-groups $ALB_SG `
  --scheme internet-facing `
  --type application `
  --region us-east-1
```

```
# Crear Target Group
```

```
aws elbv2 create-target-group `
  --name eafit-tg-web `
  --protocol HTTP `
  --port 5000 `
  --vpc-id $VPC_ID `
  --target-type ip `
  --region us-east-1
` ` `
```

```
---
```

Configuración del Dominio con DuckDNS

¿Por qué DuckDNS?

| | |
|--------------------|-------------------------------|
| Ventaja | Descripción |
| ----- | ----- |
| Gratis | Sin costo de suscripción |
| Fácil | Setup en minutos |
| Dinámico | Actualiza automáticamente IP |
| HTTPS | Soporte para certificados SSL |
| Confiable | Servicio estable desde 2010 |
| Sin límites | Ilimitados subdominios |

Paso 1: Registrarse en DuckDNS

1. Ir a <https://www.duckdns.org>
2. Click en "Sign in"
3. Autenticar con GitHub/Google
4. Aceptar permisos

Paso 2: Crear Dominio

1. En el dashboard
2. Agregar nuevo dominio (ej: `proyectoiota`)
3. Tu dominio será: `proyectoiota.duckdns.org`
4. ****Guardar el token**** (necesario para actualizaciones)

Paso 3: Apuntar al ALB

```
` `` powershell
```

```
# Obtener DNS del ALB
```

```
$ALB_DNS = aws elbv2 describe-load-balancers `
--names eafit-alb `
--region us-east-1 `
--query 'LoadBalancers[0].DNSName' `
--output text
```

```
Write-Host "ALB DNS: $ALB_DNS"
```

```
` ``
```

En el dashboard de DuckDNS:

1. Seleccionar dominio creado
2. En campo "IP", ingresar DNS del ALB
3. Click en "update ip"
4. Esperar 5-10 minutos para propagación DNS

Paso 4: Obtener Certificado SSL

```
` `` powershell
# Solicitar certificado en ACM
aws acm request-certificate `
--domain-name proyectoiota.duckdns.org `
--subject-alternative-names "*.proyectoiota.duckdns.org" `
--validation-method DNS `
--region us-east-1

# Obtener ARN del certificado
$CERT_ARN = aws acm list-certificates `
--region us-east-1 `
--query 'CertificateSummaryList[0].CertificateArn' `
```

--output text

Write-Host "Certificado ARN: \$CERT_ARN"

````

### **Validar certificado:**

1. Esperar email de ACM con instrucciones
2. Seguir enlace de validación
3. Esperar confirmación (10-15 minutos)

### **Paso 5: Crear Listener HTTPS en ALB**

```powershell

Obtener ARNs necesarios

\$ALB_ARN = aws elbv2 describe-load-balancers `

--names eafit-alb `

--region us-east-1 `

--query 'LoadBalancers[0].LoadBalancerArn' `

--output text

\$TG_ARN = aws elbv2 describe-target-groups `

--names eafit-tg-web `

--region us-east-1 `

--query 'TargetGroups[0].TargetGroupArn' `

--output text

Crear listener HTTPS

aws elbv2 create-listener `

--load-balancer-arn \$ALB_ARN `

```
--protocol HTTPS `
--port 443 `
--certificates CertificateArn=$CERT_ARN `
--default-actions Type=forward,TargetGroupArn=$TG_ARN `
--region us-east-1
` ` `
```

Verificación y Testing

Verificar Recursos en AWS

```
` ` ` powershell
# Verificar repositorios ECR
aws ecr describe-repositories --region us-east-1

# Verificar base de datos RDS
aws rds describe-db-instances `
--db-instance-identifier eafit-mysql-db `
--region us-east-1

# Verificar cluster ECS
aws ecs list-clusters --region us-east-1

# Verificar ALB
aws elbv2 describe-load-balancers `
--names eafit-alb `
--region us-east-1
```



```

## Pruebas de Aplicación

### Registrar un usuario

```bash

```
curl -X POST http://proyectoiota.duckdns.org/register \  
-H "Content-Type: application/x-www-form-urlencoded" \  
-d "nombre=Test User&comuna=Comuna 1&carrera=Ingeniería"
```

```

### Ver estadísticas

```bash

```
curl http://proyectoiota.duckdns.org/stats?password=admin123
```

```

### Enviar reporte por email

```bash

```
curl -X POST "http://proyectoiota.duckdns.org/api/send-  
report?password=admin123&email=tu-correo@example.com"
```

```

### Verificar HTTPS

```bash

```
curl https://proyectoiota.duckdns.org/es
```

```

### Ver Logs

```
` `` powershell

Logs de aplicaciones web

aws logs tail /ecs/eafit-web-es --follow --region us-east-1

Logs de estadísticas

aws logs tail /ecs/eafit-stats --follow --region us-east-1

Logs de NGINX

aws logs tail /ecs/eafit-nginx --follow --region us-east-1
` ``
```

---

## Troubleshooting

### Error: "No se puede conectar a ALB"

**Causa:** Security Group no permite tráfico o servicios no están activos

### Solución:

```
` `` powershell

Verificar Security Group

aws ec2 describe-security-groups --group-names eafit-alb-sg --region us-east-1

Verificar estado de servicios ECS

aws ecs describe-services `

--cluster eafit-cluster `

--services eafit-service `

--region us-east-1
```

```

Error: "Certificado inválido"

Causa: Certificado ACM aún en validación

Solución:

1. Esperar 10-15 minutos a validación de ACM
2. Verificar registro DNS se creó correctamente
3. Limpiar caché del navegador
4. Probar en navegador privado

Error: "Base de datos no accesible"

Causa: Endpoint, contraseña o Security Group incorrecto

Solución:

```
``` powershell
Verificar endpoint RDS
aws rds describe-db-instances `
 --db-instance-identifier eafit-mysql-db `
 --query 'DBInstances[0].Endpoint' `
 --region us-east-1

Verificar Security Group de RDS
aws ec2 describe-security-groups `
 --region us-east-1 `
 --query 'SecurityGroups[?GroupName==` default `]'
```
```

DuckDNS no actualiza IP

Causa: Token incorrecto o espera de propagación

Solución:

1. Verificar token es correcto en dashboard
2. Esperar 5-10 minutos para propagación DNS
3. Probar manualmente:

```

`https://www.duckdns.org/update?domains=proyectoiotatoken=ALB_IP`

```

Contenedores locales no inician

Causa: Puerto en uso, dependencia no lista, imagen corrupta

Solución:

```bash

# Ver logs detallados

`docker-compose logs`

# Reconstruir imágenes

`docker-compose build --no-cache`

# Iniciar con salida de depuración

`docker-compose up --verbose`

```

Seguridad

Recomendaciones de Producción

Certificados y HTTPS

- ✓ Usar HTTPS en producción (certificado ACM)
- ✓ Validar certificado en DuckDNS
- ✓ Renovación automática de certificados

Contraseñas y Credenciales

- ✓ Cambiar contraseña de admin (no usar `admin123`)
- ✓ Cambiar contraseña de RDS
- ✓ Usar app-specific password para Gmail SMTP
- ✓ Rotación periódica de contraseñas

Control de Acceso

- ✓ Security Groups restringidos
- ✓ Whitelist de IPs si es posible
- ✓ VPC endpoints para servicios privados
- ✓ IAM roles con permisos mínimos

Datos

- ✓ Backups automáticos en RDS (7 días configurados)
- ✓ Snapshots regulares de RDS
- ✓ Encriptación de datos en tránsito (SSL/TLS)
- ✓ Encriptación de datos en reposo (RDS encryption)

Monitoreo

- ✓ CloudWatch alarms para recursos
- ✓ Logs centralizados en CloudWatch
- ✓ Alertas por email en eventos críticos
- ✓ Health checks en ALB

Headers de Seguridad

...

X-Frame-Options: DENY

X-Content-Type-Options: nosniff

X-XSS-Protection: 1; mode=block

Strict-Transport-Security: max-age=31536000

...

Costos

Con AWS Educate

| Beneficio | Valor |

|-----|-----|

| **Crédito Mensual** | \$100 USD |

| **Cobertura** | 100% del proyecto |

| **Duración** | 12 meses |

| **Costo Real** | \$0 |

AWS Educate es ideal para este proyecto. Cubre completamente todos los gastos.

Optimizaciones de Costo

1. ****Auto-scaling**** en ECS (apagar en horas no pico)
2. ****Reserved instances**** para RDS (60% descuento)
3. ****Savings Plans**** para EC2 (40% descuento)
4. ****Monitoreo**** de costos en AWS Cost Explorer

Comandos Útiles

Docker Compose (Local)

```
```bash
```

```
Ver estado de contenedores
```

```
docker-compose ps
```

```
Ver logs en tiempo real
```

```
docker-compose logs -f
```

```
Ver logs de servicio específico
```

```
docker-compose logs -f nginx
```

```
docker-compose logs -f db
```

```
docker-compose logs -f stats
```

```
Ejecutar comando en contenedor
```

```
docker-compose exec db mysql -u root -p usuarios
```

```
docker-compose exec web-es bash
```

# Detener servicios

docker-compose down

# Detener y limpiar datos

docker-compose down -v

# Reconstruir imágenes

docker-compose build --no-cache

# Reiniciar servicios

docker-compose restart

docker-compose restart nginx

````

AWS CLI (Producción)

````powershell

# Verificar credenciales

aws sts get-caller-identity

# Listar recursos ECR

aws ecr list-repositories --region us-east-1

aws ecr describe-images --repository-name practicafinal-web-es --region us-east-1

# Listar instancias RDS

aws rds describe-db-instances --region us-east-1

# Listar clusters ECS



```
aws ecs list-clusters --region us-east-1
```

```
aws ecs list-services --cluster eafit-cluster --region us-east-1
```

```
Ver detalles de ALB
```

```
aws elbv2 describe-load-balancers --region us-east-1
```

```
aws elbv2 describe-target-groups --region us-east-1
```

```
Ver logs en CloudWatch
```

```
aws logs describe-log-groups --region us-east-1
```

```
aws logs tail /ecs/eafit-web-es --follow --region us-east-1
```

```
```\n
```

```
---
```

Resumen de Configuración

Archivo .env (Local)

```
```.env
```

```
DB_HOST=db
```

```
DB_USER=root
```

```
DB_PASSWORD=eafit_2025_secure
```

```
DB_NAME=usuarios
```

```
ADMIN_PASSWORD=admin123
```

```
SMTP_SERVER=smtp.gmail.com
```

```
SMTP_PORT=587
```

```
SMTP_USER=tu-email@gmail.com
```

```
SMTP_PASSWORD=tu-app-password
```

```
```\n
```

Archivo .env (Producción)

```
```.env

DB_HOST=eafit-mysql-db.XXXXX.us-east-1.rds.amazonaws.com

DB_USER=admin

DB_PASSWORD=EafitDB2025Secure!

DB_NAME=usuarios

ADMIN_PASSWORD=tu-contraseña-segura

SMTP_SERVER=smtp.gmail.com

SMTP_PORT=587

SMTP_USER=tu-email@gmail.com

SMTP_PASSWORD=tu-app-password

...
```

---

## Checklist de Despliegue

### Desarrollo Local

- Docker y Docker Compose instalados
- Proyecto clonado
- ``.env`` configurado
- Imágenes construidas (``docker-compose build``)
- Servicios iniciados (``docker-compose up -d``)
- Acceso a <http://localhost/es>
- Acceso a <http://localhost/stats?password=admin123>
- Base de datos con datos de prueba

### Despliegue AWS

- AWS CLI instalado y configurado

- Credenciales AWS válidas
- Repositorios ECR creados
- Imágenes subidas a ECR
- Base de datos RDS creada y disponible
- Cluster ECS creado
- ALB creado y configurado
- Certificado ACM solicitado y validado
- Listeners HTTP (80) y HTTPS (443) creados
- DuckDNS dominio creado
- DuckDNS apuntando a ALB
- Acceso a <https://proyectoiotaduckdns.org/es>
- Acceso a <https://proyectoiotaduckdns.org/stats?password=admin123>
- Reportes por email funcionando
- Backups RDS configurados
- CloudWatch monitoring activo

---

## **Recursos Adicionales**

### **Documentación Oficial**

- **AWS Educate:** <https://aws.amazon.com/education/awseducate/>
- **DuckDNS:** <https://www.duckdns.org/>
- **Docker:** <https://docs.docker.com/>
- **Flask:** <https://flask.palletsprojects.com/>
- **NGINX:** <https://nginx.org/en/docs/>
- **MySQL:** <https://dev.mysql.com/doc/>

### **Herramientas Útiles**

- **AWS Console:** <https://console.aws.amazon.com/>
- **CloudWatch:** <https://console.aws.amazon.com/cloudwatch/>
- **RDS Console:** <https://console.aws.amazon.com/rds/>
- **ECR Console:** <https://console.aws.amazon.com/ecr/>

## Tutoriales y Guías

- Flask + Docker
- AWS ECS Fargate deployment
- DuckDNS with AWS
- MySQL UTF-8 configuration
- Let's Encrypt certificates

---

## Conclusiones

Este sistema proporciona:

1. **Escalabilidad:** Fácil agregar más servidores web
2. **Alta disponibilidad:** Load balancing automático
3. **Seguridad:** SSL/TLS, autenticación, UTF-8
4. **Confiabilidad:** Backups automáticos, health checks
5. **Monitoreo:** CloudWatch, logs centralizados
6. **Costo efectivo:** Especialmente con AWS Educate
7. **Fácil despliegue:** Scripts automáticos disponibles

El proyecto está **listo para producción** y puede escalar según las necesidades.

